Work Division (3 People, 1-2 Days)

The project is split into three parts so that everyone can work at the same time. The parts connect through a shared contract, so agree on it first.

Shared Contract (agree on this together, first hour)

Write these down and do not change them without telling everyone.

Journey DFA

- States: `J0` Issued, `J1` Inside, `JE` Excess/Over-time Pending, `J2` Exited Normally (accepting), `J3` Exited Early (accepting), `JT` Rejected
- Symbols: `E, D, X, O, T, P, W, R`

Booking DFA

- States: `q0` to `q6`, plus `qd` (dead)
- Symbols: `src, dst, confirm, pay_ok, pay_fail, gen, cancel`

Network: one circular line, 8 stations numbered 0 to 7, trains run in one direction only (0 → 1 → ... → 7 → 0). No terminus and no forced exit.

Distance: `distance(src, x) = (x - src) mod 8`

Settings (in `config.py`)

```
NUM_STATIONS = 8
MINUTES_PER_STATION = 2.5
BUFFER_MINUTES = 1.5        # must stay below MINUTES_PER_STATION
TIME_LIMIT = 19             # (8 - 1) * 2.5 + 1.5
OVERTIME_CHARGE = 20
```

Function names

Function	Input	Output
`distance(src, dst)`	two station IDs	forward distance, 0 to 7
`fare(src, dst)`	two station IDs	fare as a number (slabs by forward distance)
`check_payment(inserted, fare)`	total cash inserted, fare	(`pay_ok` or `pay_fail`, change to return)
`create_token(src, dst)`	two station IDs	token string
`verify_token(token)`	token string	(valid or not, fields)
`classify(token, station, gate_type, now)`	token, station, "entry"/"exit", current time in minutes	one symbol
`DFA.run(string)`	string of symbols	(accepted, trace)
`process_scan(token, station, gate_type, now)`	token, station, gate type, current time	(symbol, new_state, output, message)
`pay_excess(token)`	token	(symbol `P`, new_state, output, message)

The `now` value is a plain number of minutes, so the demo can pass a simulated clock instead of real time.

Fare slabs: forward distance 1-2 = ₹10, 3-5 = ₹20, 6-7 = ₹30.

Person 1: Automata Engine (theory core)

Goal: a small, correct library that implements every Automata Theory topic in the abstract. It knows nothing about stations, tokens, time or the UI.

Tasks (in order)

1. **DFA class.** Stores states, alphabet, transition table, start state and final states. Can take one step, run a whole string, and return the full trace of states. Missing transitions go to a dead state.
2. **Define the two DFAs as data.** Fill in the Booking DFA and Journey DFA tables from the README (the Journey DFA now includes the `T` column). Write the regular expression for each:
   - Journey: `E (D | X | (O | T)+ P)`
   - Booking: `src (src | dst)* dst confirm pay_fail* pay_ok gen`
3. **Moore and Mealy machines.** One class for each. Moore attaches an output to every state, Mealy to every transition. Both return the list of outputs for an input string. Use the gate open/locked tables from the README, including the new `J1 --T/...--> JE` transition.
4. **NFA class and subset construction.** Use the payment-method example (UPI, card, cash branching and merging at Paid). This is a theory demonstration only, because the kiosk accepts cash. Implement NFA to DFA conversion and keep the intermediate table (which set of NFA states becomes which DFA state), since the report needs it.
5. **DFA minimization (table-filling).** Build a deliberately redundant Booking DFA (for example `PaidCash` and `PaidUPI` behaving the same). The function returns the merged DFA and the marking table round by round, so the UI can show "8 states to 7 states" and why.
6. **Test file.** Run all sequences from the test table (`ED` , `EX` , `EOP` , `ETP` , `EOOP` , `EO` , `ET` , `EE` , `D` , `EDR`) and check accept or reject.

Hands over to others

- To Person 2: the Journey DFA object, ready to call `step(state, symbol)` .
- To Person 3: functions that return tables and traces, such as `get_transition_table()` , `get_subset_steps()` , `get_minimization_steps()` , plus the regex strings.

Done when

- All test strings give the expected result.
- Subset construction output matches a by-hand check.
- Minimization reduces the redundant DFA to the expected size.

Viva topics

- Why a missing transition goes to the dead state.
- How subset construction works.

- Why two states are equivalent (Myhill–Nerode).
- Difference between Moore and Mealy outputs.
- Why `O` and `T` both lead to the same Excess Pending state.

Person 2: Ticket, Gate and Time Logic

Goal: everything that makes the system behave like a metro. It turns real-world events (tickets, scans, time) into the symbols that Person 1's DFA understands.

Tasks (in order)

1. **Settings and stations.** Create `config.py` with the settings above. Make a list of 8 stations with one global ID each.
2. **Distance and fare.** `distance(src, dst) = (dst - src) mod 8`. `fare` uses the slabs on the distance. Also an `excess_fare(src, exit_station, paid_fare)` function that returns the fare for the distance to the exit station minus the paid fare. And `check_payment(inserted, fare)`: if `inserted >= fare` it returns `pay_ok` and the change (`inserted - fare`), otherwise `pay_fail` and no change. The kiosk calls this each time cash is inserted.
3. **Token creation and verification.** Token = pair code (source ID + destination ID) + transaction ID + timestamp + HMAC signature. `verify_token` must reject any edited token. Add QR image generation if time allows.
4. **Shared record (the "central server").** A dictionary or small SQLite table: `txn_id -> {state, source, dest, paid_fare, entry_time}`. It answers "is this ticket inside or already closed, and when did it enter?".
5. **Classifier.** Takes a token, the scan station, the gate type and the current time, and returns one symbol:
   - Entry gate: bad signature → `W`; closed ticket → `R`; unused and at the source → `E` (and save `entry_time`); otherwise `W`
   - Exit gate: bad signature → `W`; closed ticket → `R`; not inside → `W`; then in this order:
     1. `now - entry_time` greater than `TIME_LIMIT` → `T`
     2. exit station is the destination → `D`
     3. `distance(source, exit)` less than `distance(source, destination)` → `X`
     4. otherwise → `O`
6. **`process_scan` and `pay_excess`.** `process_scan` calls the classifier, feeds the symbol to the Journey DFA using the ticket's current state, saves the new state, and returns the gate output (OPEN/LOCKED, amount to pay, message). The amount is the excess fare for `O` and `OVERTIME_CHARGE` for `T`. `pay_excess` feeds `P` into the DFA.
7. **Tests.** Use the situation table in the README (ticket 0 to 3 and the wrap-around ticket 6 to 1), plus edited tokens, wrong entry station, double entry, reuse after exit, and over-travel or over-time followed by payment.

Needs from others

Person 1's DFA object. Until it is ready, use a plain dictionary of transitions as a stand-in.

Done when

Every row of the test tables gives the expected symbol, new state and gate output, including the wrap-around cases.

Viva topics

- Why the token is signed, and why the pair code alone is not enough.
- How over-travel and early exit are detected by comparing forward distances.
- Why the time limit is needed on a loop (the wrap-around ambiguity) and how the 19 minute limit is chosen.
- Why the buffer must be shorter than the time between two stations.
- Why the classifier exists (a DFA cannot compare numbers or time).

Person 3: UI, Integration and Report

Goal: make the project visible, connected and well explained. This person decides how good the demo looks and how well the report reads.

Tasks (in order)

1. **Start immediately with stubs.** On day 1, write fake versions of the shared functions (for example `fare()` always returns 20, `classify()` always returns `D`) so the UI can be built before the other parts exist. Swap in the real functions on day 2.
2. **Screens** (Streamlit is quickest):
   - **Kiosk:** source and destination dropdowns, a **Show Fare** button, a **Proceed to Pay** button (two separate `confirm` steps in the Booking DFA), a cash insertion area (amount box or ₹10 / ₹20 / ₹50 buttons) with a running total that calls `check_payment` each time, shows "₹__ more needed" or the change, ticket generation, token or QR shown. A side panel shows the Booking DFA state updating as the user goes through.
   - **Gate simulator:** station dropdown, entry/exit toggle, token box, a **"minutes since entry"** input (the simulated clock, passed as `now`), and a Scan button. Shows the symbol, state change, Moore/Mealy output and an OPEN/LOCKED indicator. When locked, show a "Pay" button for the excess or over-time charge. Optionally add a history line of the symbols so far for the current ticket (for example `E O`) so it can be copied into the string tester.
   - **String tester:** type a sequence such as `EOP` or `ETP` and see accept or reject with the full trace.
   - **Automata viewer:** transition tables, regexes, NFA to DFA steps, and a Minimize button with before and after.

3. **Diagrams.** Draw the Booking DFA, Journey DFA (with `T`), NFA, subset-constructed DFA, minimized DFA, and the Moore and Mealy versions in draw.io. Also draw the circular route with the 8 stations and the direction of travel. They are needed for the report anyway.
4. **Report.** Write sections for: introduction, network (circular line, forward distance) and fare, token design, Booking DFA, Journey DFA, over-travel, over-time and early exit, the time limit and how it is set, NFA to DFA, minimization, Moore and Mealy, test cases, limitations, future work.
5. **Demo script.** A fixed order, for example: book a ticket (insert too little cash first, then enough) → scan entry → normal exit → try reuse (rejected) → early exit → over-travel and pay excess → full loop with the clock set to 20 minutes (over-time) and pay → edited token → string tester → minimization.
6. **Viva preparation.** Collect likely questions and make sure each member can answer questions about their own part.

Needs from others

Person 1's table and step functions, and Person 2's `process_scan`, `pay_excess` and token functions.

Done when

The demo script runs start to finish without errors, and every diagram matches the actual tables in the code.

Viva topics

- How the whole system fits together.
- What each screen shows, and why the simulated clock is used.
- The limitations (single circular line, no real camera, time used as a stand-in for distance).

Timeline

Time	Person 1	Person 2	Person 3
Day 1 start	Agree on the contract together	Agree on the contract together	Agree on the contract together
Day 1	DFA, Moore/Mealy, then NFA	Config, distance, fare, token, classifier	UI with stubs, begin diagrams
Day 1 evening	Minimization and tests	Shared record, time check and `process_scan`	Finish screens
Day 2 morning	Hand over final functions	Hand over final functions	Integrate and fix bugs
Day 2 afternoon	Help with report sections on own topics	Same	Finish report, rehearse demo

---

Two Rules to Avoid Problems

1. **Do not change the contract alone.** If a state, symbol, setting or function name changes, tell everyone, because three codebases depend on it.
2. **Cross-check on Day 2.** Each person explains one other person's part to the group. This finds misunderstandings and prepares everyone for viva questions on any part of the project.